

HirePulse Job Portal

Code explained line by line

Files covered: index.html, styles.css and script.js

Every line or group of lines is shown next to a plain-language explanation of what it does.

How to read this document

Each part has a table with three columns: **Lines** (the line numbers in your file), **Code** (the exact code from your file) and **What it does** (the explanation). Where several lines do one job together (for example, one CSS rule or one function), they are explained as a group. Small pieces of code inside explanations look like this.

How the three files work together

index.html is the skeleton: it defines every piece of the page (forms, buttons, popups, empty containers). **styles.css** is the skin: it controls colours, spacing and layout. **script.js** is the brain: it reacts to clicks and typing, saves data in the browser, and builds the job cards and tables by writing HTML into the empty containers. The HTML connects to the JavaScript through attributes such as `onclick="handleLogout()"` and through `id` values that the JavaScript uses to find elements with `document.getElementById(...)`.

What happens when the page opens

1) The browser reads index.html and loads the styles and the script. 2) At first only the login screen is visible, because the navbar and main area have the `hidden` class. 3) script.js runs `restoreSession()`: if you were logged in before, the portal opens; otherwise the login screen stays. 4) After login, `render()` shows either the job seeker view or the employer dashboard, and every action (search, bookmark, apply, post a job) updates the saved data and redraws the screen.

Key words explained

Element / tag	A piece of HTML such as <code><h1></code> or <code><button></code> . Most have an opening and closing tag.
Attribute	Extra information inside a tag, such as <code>type="email"</code> or <code>class="btn"</code> .
id vs class	An id names one unique element (JavaScript uses it to find it). A class can be shared by many elements and is mainly used by CSS.
CSS selector	The part before <code>{ }</code> that says which elements to style: <code>.name</code> (class), <code>#name</code> (id), <code>name</code> (tag), <code>a</code> , <code>b</code> (list), <code>a b</code> (inside).
Flexbox / Grid	Two CSS layout systems. Flexbox arranges items in a row or column; Grid arranges them in rows and columns.
CSS variable	A reusable value such as <code>--primary</code> , defined once and used with <code>var(--primary)</code> .
Media query	A CSS block (<code>@media</code>) that applies only on certain screen sizes, used for mobile layouts.
Event / handler	An event is something that happens (click, typing, submit). A handler is the function that runs, for example <code>onclick="openModal(...)"</code> .
DOM	The live tree of page elements that JavaScript can read and change with <code>document.getElementById</code> , <code>classList</code> , <code>innerHTML</code> and so on.
const / let	Ways to create variables. <code>const</code> cannot be reassigned; <code>let</code> can.
Array / object	An array is a list <code>[a, b, c]</code> . An object groups named values <code>{ title: "X", salary: "Y" }</code> .
Arrow function	A short function form: <code>job => job.id</code> means "take a job and return its id".
Template literal	Text in back-tick quotes that can insert values with <code>\${...}</code> and span several lines.
localStorage / JSON	<code>localStorage</code> keeps text in the browser even after refresh. <code>JSON.stringify</code> turns data into text to save it and <code>JSON.parse</code> turns it back.
async / await	Lets code wait for a slow task (like hashing a password or reading a file) without freezing the page.

Part 1: index.html (the structure)

The HTML file defines what is on the page. Everything dynamic (job cards, table rows, applicant cards) starts as an empty container here and is filled in by script.js.

Lines	Code	What it does
Page setup (the head)		
1	<code><!DOCTYPE html></code>	Declares that this is an HTML5 document, so the browser uses modern standards mode instead of old "quirks" mode.
2	<code><html lang="en"></code>	The root element that wraps the whole page. <code>lang="en"</code> says the content is English; this helps screen readers, search engines and translation tools.
3	<code><head></code>	Opens the head: information about the page that is not drawn directly on screen.
4	<code><meta charset="UTF-8" /></code>	Sets the character encoding to UTF-8 so symbols such as ₹, — and × show correctly.
5	<code><meta name="viewport" content="width=device-width, initial-scale=1.0" /></code>	Makes the page responsive on phones: the page width matches the device width and the starting zoom is 1 (no tiny zoomed-out desktop view).
6	<code><title>HirePulse – Modern Job Portal</title></code>	The text shown on the browser tab and in bookmarks or search results.
7	<code><link rel="stylesheet" href="styles.css" /></code>	Connects your own stylesheet <code>styles.css</code> , which controls colours and layout. The file must be in the same folder and spelled exactly the same.
8–9	<code><!-- FontAwesome for clean UI icons --> <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css" /></code>	A comment (ignored by the browser) and a link to the Font Awesome icon library on a CDN. Icons like <code><i class="fa-solid fa-briefcase"></code> come from here, so this needs an internet connection.
10–11	<code></head> <body></code>	Closes the head and opens the body, which holds everything people can see.
Login / sign-up screen		
13–14	<code><!-- AUTH SCREEN (Login / Sign up) --> <section id="authScreen" class="auth-screen"></code>	A comment, then a <code><section></code> with <code>id="authScreen"</code> . The id lets JavaScript find it and the class <code>auth-screen</code> lets CSS style it as a full-screen, two-column layout. It is visible first and hidden after login.
15	<code><div class="auth-aside"></code>	<code>auth-aside</code> : the dark left column with branding and a headline.
16–19	<code><div class="nav-brand auth-brand"> <i class="fa-solid fa-briefcase brand-icon"></i> HirePulse </div></code>	The logo: a briefcase icon (an <code><i></code> tag with Font Awesome classes) and the name HirePulse inside a <code></code> . The classes <code>nav-brand</code> <code>auth-brand</code> reuse the navbar logo style and add a small auth tweak.
20–21	<code><h1>Jobs across India, all in one place.</h1> <p>Browse openings in Bengaluru, Hyderabad, Pune and more, or post a role and review applicants.</p></code>	The main headline (<code><h1></code>) and a one-line description (<code><p></code>) for new visitors.

Lines	Code	What it does
22–26	<pre> <ul class="auth-points"> <i class="fa-solid fa-circle-check"></i> Internships and fresher roles <i class="fa-solid fa-circle-check"></i> Apply with your PDF resume </pre>	A bullet list: <code></code> is the list and each <code></code> is one item with a green tick icon. Line 23 is an empty line left where one item was removed.
27	<pre> </div> </pre>	Closes the left column.
29–30	<pre> <div class="auth-panel"> <div class="auth-card"> </pre>	<code>auth-panel</code> is the right column that centres the form. <code>auth-card</code> is the white box that holds the tabs and both forms.
31–34	<pre> <div class="auth-tabs"> <button type="button" id="tabLogin" class="auth-tab active" onclick="switchAuthTab('login')">Log in</button> <button type="button" id="tabSignup" class="auth-tab" onclick="switchAuthTab('signup')">Sign up</button> </div> </pre>	The Log in / Sign up switch. Two buttons with <code>type="button"</code> so they never submit a form. <code>onclick="switchAuthTab('login')"</code> runs that JavaScript function when clicked. The first button has class <code>active</code> , so it looks selected.
36	<pre> <div id="authError" class="auth-error hidden" role="alert"></div> </pre>	A red error box, hidden at the start by the <code>hidden</code> class. JavaScript puts messages like "Incorrect email or password." in it. <code>role="alert"</code> makes screen readers announce the message.

Login form

38–39	<pre> <!-- Login form --> <form id="loginForm" onsubmit="handleLogin(event)"> </pre>	A comment and the login <code><form></code> . <code>onsubmit="handleLogin(event)"</code> calls this function when the user clicks Log in or presses Enter.
40–41	<pre> <h2 class="auth-title">Welcome back</h2> <p class="auth-sub">Log in to continue to your dashboard.</p> </pre>	The form heading (<code><h2></code>) and a short helper sentence.
42–45	<pre> <div class="form-group"> <label for="loginEmail">Email address</label> <input type="email" id="loginEmail" required autocomplete="email" placeholder="aarav@example.com" /> </div> </pre>	Email field. The <code><label for="loginEmail"></code> is linked to the input by its id, so clicking the label focuses the input. <code>type="email"</code> checks the format, <code>required</code> blocks an empty submit, <code>autocomplete="email"</code> lets the browser autofill, and <code>placeholder</code> is the grey hint text.
46–54	<pre> <div class="form-group"> <label for="loginPassword">Password</label> <div class="password-field"> <input type="password" id="loginPassword" required autocomplete="current-password" placeholder="Your password" /> <button type="button" class="toggle-pw" onclick="togglePassword('loginPassword', this)" aria-label="Show password"> <i class="fa-regular fa-eye"></i> </button> </div> </div> </pre>	Password field inside a <code>password-field</code> wrapper, so the eye button can sit inside the box. <code>type="password"</code> hides the characters. The eye button calls <code>togglePassword('loginPassword', this)</code> to show or hide the text. <code>aria-label</code> describes the icon-only button for screen readers.

Lines	Code	What it does
55	<pre><button type="submit" class="btn btn-primary auth-submit">Log in</button></pre>	The submit button. <code>type="submit"</code> triggers the form's <code>onsubmit</code> . The three classes <code>btn btn-primary auth-submit</code> combine three styles.
56	<pre><p class="auth-switch">New to HirePulse? <button type="button" class="link-btn" onclick="switchAuthTab('signup')">Create an account</button></p></pre>	Helper text with a link-style button that switches to the sign-up tab.
57	<pre></form></pre>	Closes the login form.
Sign-up form		
59–60	<pre><!-- Sign up form --> <form id="signupForm" class="hidden" onsubmit="handleSignup(event)"></pre>	A comment and the sign-up form. It starts with the class <code>hidden</code> , so only the login form shows at first. <code>onsubmit</code> calls <code>handleSignup</code> .
61–62	<pre><h2 class="auth-title">Create your account</h2> <p class="auth-sub">Takes less than a minute.</p></pre>	Heading and helper text.
63–70	<pre><div class="form-group"> <label for="signupName">Full name</label> <input type="text" id="signupName" required autocomplete="name" placeholder="Aarav Sharma" /> </div> <div class="form-group"> <label for="signupEmail">Email address</label> <input type="email" id="signupEmail" required autocomplete="email" placeholder="aarav@example.com" /> </div></pre>	The full name field (<code>type="text"</code> , <code>autocomplete="name"</code>) and the email field. They are built the same way as the login fields.
71–73	<pre><div class="role-picker"> I want to <div class="role-options"></pre>	The role picker. The <code></code> is the small title "I want to". <code>role-options</code> places the two choices side by side.
74–77	<pre> <label class="role-option"> <input type="radio" name="signupRole" value="seeker" checked /> <i class="fa-solid fa-magnifying-glass"></i> Find a job </label></pre>	First choice: a <code><label></code> wrapped around a radio input. <code>name="signupRole"</code> groups the radios so only one can be selected. <code>value="seeker"</code> is what JavaScript reads. <code>checked</code> makes it the default. The <code></code> is the visible card (CSS hides the real radio circle).
78–81	<pre> <label class="role-option"> <input type="radio" name="signupRole" value="employer" /> <i class="fa-solid fa-building"></i> Hire talent </label></pre>	Second choice, same structure, with <code>value="employer"</code> ("Hire talent").
82–83	<pre></div> </div></pre>	Closes the options and the picker.

Lines	Code	What it does
84–92	<pre> <div class="form-group"> <label for="signupPassword">Password</label> <div class="password-field"> <input type="password" id="signupPassword" required minlength="8" autocomplete="new-password" placeholder="At least 8 characters" /> <button type="button" class="toggle-pw" onclick="togglePassword('signupPassword', this)" aria-label="Show password"> <i class="fa-regular fa-eye"></i> </button> </div> </div> </pre>	Password field with an eye toggle. <code>minlength="8"</code> asks for at least 8 characters. <code>autocomplete="new-password"</code> tells the browser this is a new password, so it may suggest a strong one.
93–96	<pre> <div class="form-group"> <label for="signupConfirm">Confirm password</label> <input type="password" id="signupConfirm" required autocomplete="new-password" placeholder="Re-enter your password" /> </div> </pre>	Confirm-password field, so typing mistakes are caught (JavaScript compares the two values).
97–98	<pre> <button type="submit" class="btn btn-primary auth-submit">Create account</button> <p class="auth-switch">Already have an account? <button type="button" class="link-btn" onclick="switchAuthTab('login')">Log in</button></p> </pre>	The Create account button and a link back to the login tab.
99–102	<pre> </form> </div> </div> </section> </pre>	Closes the form, the card, the panel and the whole auth section.
Navigation bar and toast area		
104–105	<pre> <!-- Top Navigation Header --> <header class="navbar hidden" id="appHeader"> </pre>	A comment and the header. It has the classes <code>navbar</code> <code>hidden</code> and <code>id="appHeader"</code> ; it stays hidden until login, when JavaScript removes <code>hidden</code> .
106–109	<pre> <div class="nav-brand"> <i class="fa-solid fa-briefcase brand-icon"></i> HirePulse </div> </pre>	The logo on the left side of the navbar.
111	<pre> <div class="nav-controls"> </pre>	<code>nav-controls</code> : the container for the items on the right.
112–118	<pre> <div class="user-chip"> <div class="avatar" id="userAvatar">?</div> <div class="user-meta"> <strong id="userNameLabel"> <small id="userRoleLabel"></small> </div> </div> </pre>	The user chip: a circle (<code>userAvatar</code>) with your first initial, your name (<code>userNameLabel</code>) and your role (<code>userRoleLabel</code>). The <code>?</code> and the empty tags are placeholders that JavaScript replaces after login.

Lines	Code	What it does
119–121	<pre><button class="btn btn-secondary btn-sm" onclick="resetPortalData()"> <i class="fa-solid fa-rotate-left"></i> Reset Demo Data </button></pre>	The Reset Demo Data button. onclick runs resetPortalData() . The <i> is a rotate-left icon.
122–124	<pre><button class="btn btn-secondary btn-sm" onclick="handleLogout()"> <i class="fa-solid fa-right-from-bracket"></i> Log out </button></pre>	The Log out button. onclick runs handleLogout() .
125–126	<pre></div> </header></pre>	Closes the controls and the header.
128–129	<pre><!-- Notification Toast Container --> <div id="toastContainer" class="toast-container"></div></pre>	An empty container where JavaScript adds small popup messages (toasts) such as "Job saved!". They disappear after 3 seconds.
Main area: job seeker view		
131–132	<pre><!-- Main Content Layout --> <main class="container hidden" id="appMain"></pre>	A comment and <main id="appMain" class="container hidden"> : the main content area. It is hidden until login.
134–135	<pre><!-- JOB SEEKER VIEW --> <section id="seekerSection" class="portal-section"></pre>	seekerSection : everything a job seeker sees. JavaScript shows or hides it depending on the account role.
136–138	<pre><div class="hero-banner"> <h2>Find Your Next Career Move</h2> <p>Discover opportunities from top tech companies and innovative startups.</p></pre>	The dark hero banner with a heading and a tagline.
140–141	<pre><!-- Search & Multi-Filter Bar --> <div class="search-box"></pre>	A comment and search-box , the white bar that holds the search field and the filters.
142–145	<pre><div class="input-icon-group"> <i class="fa-solid fa-magnifying-glass"></i> <input type="text" id="searchInput" placeholder="Search by job title, skill, company or city..." oninput="handleSearchAndFilter()" /> </div></pre>	Search input with a magnifier icon. oninput="handleSearchAndFilter()" runs on every keystroke, so the results update live.
147–154	<pre><select id="categoryFilter" onchange="handleSearchAndFilter()"> <option value="all">All Categories</option> <option value="Engineering">Engineering</option> <option value="Design">Design</option> <option value="Marketing">Marketing</option> <option value="Product">Product</option> <option value="Data & AI">Data & AI</option> </select></pre>	The category dropdown (<select>). value="all" means no filtering. onchange re-filters when another option is picked. &amp; is how HTML writes the & symbol.
156–162	<pre><select id="typeFilter" onchange="handleSearchAndFilter()"> <option value="all">All Job Types</option> <option value="Full-time">Full-time</option> <option value="Remote">Remote</option> <option value="Contract">Contract</option> <option value="Internship">Internship</option> </select></pre>	The job type dropdown (Full-time, Remote, Contract, Internship) with the same idea.

Lines	Code	What it does
163–164	<pre> </div> </div> </pre>	Closes the search box and the hero banner.
166–168	<pre> <!-- Stats Bar & Controls --> <div class="view-header"> Loading jobs... </pre>	A comment and <code>view-header</code> . The span <code>jobCountBadge</code> shows text like "Showing 12 of 12 jobs"; JavaScript replaces "Loading jobs...".
169–177	<pre> <div class="tab-filters"> <button id="tabAllJobs" class="tab-btn active" onclick="switchSeekerTab('all')">All Listings</button> <button id="tabSavedJobs" class="tab-btn" onclick="switchSeekerTab('saved')"> <i class="fa-solid fa-bookmark"></i> Bookmarked (0) </button> <button id="tabMyApps" class="tab-btn" onclick="switchSeekerTab('applied')"> <i class="fa-solid fa-paper-plane"></i> My Applications (0) </button> </div> </pre>	The three tab buttons. <code>onclick="switchSeekerTab('all')"</code> (or <code>'saved'</code> , <code>'applied'</code>) changes which jobs are listed. The small spans <code>savedCount</code> and <code>appliedCount</code> are updated by JavaScript.
178	<pre> </div> </pre>	Closes the view header.
180–182	<pre> <!-- Jobs Grid --> <div id="jobGrid" class="cards-grid"></div> </section> </pre>	A comment, the empty <code>jobGrid</code> div where JavaScript places one card per job, and the end of the seeker section.
Employer dashboard		
184–185	<pre> <!-- EMPLOYER VIEW --> <section id="employerSection" class="portal-section hidden"> </pre>	A comment and <code>employerSection</code> . It is hidden by default and only employer accounts see it.
186–194	<pre> <div class="dashboard-header"> <div> <h2>Employer Dashboard</h2> <p>Manage job openings, post new positions, and evaluate applicant profiles.</p> </div> <button class="btn btn-primary" onclick="openModal('postJobModal')"> <i class="fa-solid fa-plus"></i> Post New Position </button> </div> </pre>	The dashboard header: title and description on the left, and the Post New Position button on the right, which calls <code>openModal('postJobModal')</code> to open the form popup.

Lines	Code	What it does
196–204	<pre> <!-- Overview Cards --> <div class="stats-grid"> <div class="stat-card"> <i class="fa-solid fa-folder-open stat-icon text-blue"></i> <div> <h3 id="statTotalJobs">0</h3> <p>Active Postings</p> </div> </div> </div> </pre>	A comment, the <code>stats-grid</code> container and the first stat card: a folder icon, a number (<code>statTotalJobs</code> , updated by JavaScript) and the label "Active Postings".
205–212	<pre> <div class="stat-card"> <i class="fa-solid fa-users stat-icon text-green"></i> <div> <h3 id="statTotalApps">0</h3> <p>Total Applicants</p> </div> </div> </pre>	The second stat card for total applicants (<code>statTotalApps</code>), then the closing tag of the grid.
214–216	<pre> <!-- Job Management & Applicant Pipeline Table --> <div class="table-card"> <h3>Active Postings & Candidate Pipeline</h3> </pre>	A comment, the <code>table-card</code> box and its heading.
217–218	<pre> <div class="table-wrapper"> <table class="data-table"> </pre>	<code>table-wrapper</code> lets the table scroll sideways on small screens, then the <code><table></code> opens.
219–227	<pre> <thead> <tr> <th>Job Title</th> <th>Category</th> <th>Type</th> <th>Applicants</th> <th>Actions</th> </tr> </thead> </pre>	The table head: five column titles (<code><th></code> cells inside one <code><tr></code> row).
228–230	<pre> <tbody id="employerJobTableBody"> <!-- Dynamically populated --> </tbody> </pre>	<code><tbody id="employerJobTableBody"></code> : JavaScript fills it with one row per job.
231–236	<pre> </table> </div> </div> </section> </main> </pre>	Closes the table, wrapper, card, employer section and <code></main></code> .
Popups (modals)		
238–240	<pre> <!-- MODAL 1: Post Job Modal --> <div id="postJobModal" class="modal-overlay hidden"> <div class="modal-content"> </pre>	Modal 1 (post a job). <code>modal-overlay</code> is the dark full-screen backdrop, hidden by default. <code>modal-content</code> is the white dialog box.

Lines	Code	What it does
241–244	<pre> <div class="modal-header"> <h3>Post New Opportunity</h3> <button class="close-btn" onclick="closeModal('postJobModal')">&times;</button> </div> </pre>	Dialog header with a title and the × close button (×) that calls <code>closeModal('postJobModal')</code> .
245	<pre> <form id="postJobForm" onsubmit="handlePostJob(event)"> </pre>	The post-job form. <code>onsubmit</code> calls <code>handlePostJob</code> .
246–249	<pre> <div class="form-group"> <label for="jobTitle">Job Title *</label> <input type="text" id="jobTitle" required placeholder="e.g. Senior Frontend Developer" /> </div> </pre>	Job title input (required).
250–254	<pre> <div class="form-row"> <div class="form-group"> <label for="companyName">Company Name *</label> <input type="text" id="companyName" required placeholder="e.g. Zoho, Infosys, Acme Technologies Pvt. Ltd." /> </div> </pre>	<code>form-row</code> places two fields side by side. First one: company name, with Indian example names in the placeholder.
255–265	<pre> <div class="form-group"> <label for="jobCategory">Category *</label> <select id="jobCategory" required> <option value="Engineering">Engineering</option> <option value="Design">Design</option> <option value="Marketing">Marketing</option> <option value="Product">Product</option> <option value="Data & AI">Data & AI</option> </select> </div> </div> </pre>	Second field of the row: the category dropdown (includes Data & AI). The last lines close the select, the group and the row.
266–275	<pre> <div class="form-row"> <div class="form-group"> <label for="jobType">Type *</label> <select id="jobType" required> <option value="Full-time">Full-time</option> <option value="Remote">Remote</option> <option value="Contract">Contract</option> <option value="Internship">Internship</option> </select> </div> </pre>	Next row. First field: the job type dropdown.
276–280	<pre> <div class="form-group"> <label for="jobSalary">Salary Range *</label> <input type="text" id="jobSalary" required placeholder="e.g. ₹12 - 18 LPA" /> </div> </div> </pre>	Second field of the row: salary range, with a ₹ LPA example. LPA means "lakhs per annum".

Lines	Code	What it does
281–283	<pre> <div class="form-group"> <label for="jobLocation">Location *</label> <input type="text" id="jobLocation" list="cityList" required placeholder="e.g. Bengaluru, Karnataka or Remote (India)" /> </pre>	Location group: a label and an input. <code>list="cityList"</code> connects the input to the suggestion list below.
284–296	<pre> <datalist id="cityList"> <option value="Bengaluru, Karnataka"></option> <option value="Hyderabad, Telangana"></option> <option value="Pune, Maharashtra"></option> <option value="Mumbai, Maharashtra"></option> <option value="Gurugram, Haryana"></option> <option value="Noida, Uttar Pradesh"></option> <option value="Chennai, Tamil Nadu"></option> <option value="Kolkata, West Bengal"></option> <option value="Ahmedabad, Gujarat"></option> <option value="Remote (India)"></option> </datalist> </div> </pre>	The <code><datalist></code> holds city suggestions shown while typing; users can still type any value. Line 296 closes the group.
297–300	<pre> <div class="form-group"> <label for="jobDescription">Description *</label> <textarea id="jobDescription" rows="4" required placeholder="Describe responsibilities and requirements..."></textarea> </div> </pre>	Description <code><textarea></code> (a multi-line input). <code>rows="4"</code> sets its starting height.
301–304	<pre> <div class="modal-footer"> <button type="button" class="btn btn-secondary" onclick="closeModal('postJobModal')">Cancel</button> <button type="submit" class="btn btn-primary">Publish Posting</button> </div> </pre>	Footer buttons: Cancel (<code>type="button"</code> , closes the popup) and Publish Posting (<code>type="submit"</code> , sends the form).
305–307	<pre> </form> </div> </div> </pre>	Closes the form, the dialog and the overlay.
309–311	<pre> <!-- MODAL 2: Apply to Job Modal --> <div id="applyModal" class="modal-overlay hidden"> <div class="modal-content"> </pre>	Modal 2 (apply for a job): comment, overlay and dialog.
312–315	<pre> <div class="modal-header"> <h3 id="applyModalTitle">Apply for Job</h3> <button class="close-btn" onclick="closeModal('applyModal')">&times;</button> </div> </pre>	Header with <code><h3 id="applyModalTitle"></code> (JavaScript changes it to "Apply for [job] at [company]") and the close button.
316–317	<pre> <form id="applyForm" onsubmit="handleApplySubmit(event)"> <input type="hidden" id="applyJobId" /> </pre>	The apply form calls <code>handleApplySubmit</code> . The hidden input <code>applyJobId</code> stores which job is being applied to.
318–321	<pre> <div class="form-group"> <label for="applicantName">Full Name *</label> <input type="text" id="applicantName" required placeholder="Aarav Sharma" /> </div> </pre>	Full name input (JavaScript pre-fills it from your account).

Lines	Code	What it does
322–325	<pre> <div class="form-group"> <label for="applicantEmail">Email Address *</label> <input type="email" id="applicantEmail" required readonly placeholder="aarav@example.com" /> </div> </pre>	Email input with readonly : it is filled from your account and cannot be edited.
326–329	<pre> <div class="form-group"> <label for="coverLetter">Cover Letter / Note</label> <textarea id="coverLetter" rows="3" placeholder="Explain briefly why you are a great fit..."></textarea> </div> </pre>	Cover letter textarea. It has no required , so it is optional.
330–334	<pre> <div class="form-group"> <label for="resumeUpload">Upload Resume (PDF - Max 2MB) *</label> <input type="file" id="resumeUpload" accept=".pdf" required /> <small class="help-text">File will be converted and stored securely in browser storage.</small> </div> </pre>	Resume upload. type="file" opens a file picker, accept=".pdf" limits the choice to PDFs, required makes it compulsory. The <small> tag is helper text.
335–338	<pre> <div class="modal-footer"> <button type="button" class="btn btn-secondary" onclick="closeModal('applyModal')">Cancel</button> <button type="submit" class="btn btn-primary">Submit Application</button> </div> </pre>	Footer with Cancel and Submit Application buttons.
339–341	<pre> </form> </div> </div> </pre>	Closes the form, the dialog and the overlay.
343–345	<pre> <!-- MODAL 3: View Applicants Modal (Employer) --> <div id="viewApplicantsModal" class="modal-overlay hidden"> <div class="modal-content modal-lg"> </pre>	Modal 3 (view applicants): comment, overlay and a dialog with the extra class modal-lg , which makes it wider.
346–349	<pre> <div class="modal-header"> <h3 id="applicantsModalTitle">Applicants for Position</h3> <button class="close-btn" onclick="closeModal('viewApplicantsModal')">&times;</button> </div> </pre>	Header with a title (applicantsModalTitle , changed by JavaScript) and the close button.
350–354	<pre> <div class="modal-body"> <div id="applicantsListContainer" class="applicant-cards-list"> <!-- Applicants rendered here --> </div> </div> </pre>	The modal body and applicantsListContainer : an empty div where JavaScript adds one card per applicant.
355–356	<pre> </div> </div> </pre>	Closes the dialog and the overlay.

Loading the JavaScript

Lines	Code	What it does
358–360	<pre><script src="script.js"></script> </body> </html></pre>	Loads <code>script.js</code> . It sits at the very bottom so that all the HTML above already exists when the code runs. Then the body and the html tags are closed.

Part 2: styles.css (the design)

The CSS file controls how everything looks. A rule has a selector (which elements) and properties (what to change), written as selector { property: value; }.

Lines	Code	What it does
Variables and base styles		
1–16	<pre>:root { --primary: #2563eb; --primary-hover: #1d4ed8; --secondary: #64748b; --bg-main: #f8fafc; --card-bg: #ffffff; --text-dark: #0f172a; --text-muted: #64748b; --border: #e2e8f0; --accent-green: #10b981; --accent-red: #ef4444; --accent-yellow: #f59e0b; --shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05), 0 2px 4px -1px rgba(0, 0, 0, 0.03); --shadow-lg: 0 10px 15px -3px rgba(0, 0, 0, 0.1); --radius: 8px; }</pre>	:root means the top of the page. Names starting with -- are CSS variables that you reuse with var(--name); change a colour once here and it changes everywhere. Defined here: primary blue and its hover shade, secondary grey, page background, card background, dark and muted text colours, border colour, green/red/yellow accents, two shadow styles and the standard corner radius (8px).
18–23	<pre>* { box-sizing: border-box; margin: 0; padding: 0; font-family: 'Segoe UI', system-ui, -apple-system, sans-serif; }</pre>	* selects every element. box-sizing: border-box makes padding and border count inside the width, which keeps sizes predictable. margin: 0; padding: 0 removes browser default spacing. font-family lists fonts in order; the browser uses the first one available.
25–30	<pre>body { background-color: var(--bg-main); color: var(--text-dark); line-height: 1.5; padding-bottom: 50px; }</pre>	The body: page background and text colour from the variables, line-height: 1.5 for comfortable reading, and 50px of space at the bottom.
32–36	<pre>.container { max-width: 1100px; margin: 25px auto; padding: 0 20px; }</pre>	.container (a dot means class): content is at most 1100px wide. margin: 25px auto puts 25px above and below and centres it horizontally; padding: 0 20px gives 20px on the left and right.
38	<pre>.hidden { display: none !important; }</pre>	.hidden hides any element with display: none. !important beats other display rules, so adding or removing this class from JavaScript always works.
40	<pre>/* Navbar */</pre>	A comment marking the Navbar section.

Lines	Code	What it does
41–49	<pre>.navbar { background-color: var(--card-bg); border-bottom: 1px solid var(--border); padding: 15px 30px; display: flex; justify-content: space-between; align-items: center; box-shadow: var(--shadow); }</pre>	.navbar : white background, a thin bottom border, padding, a soft shadow. display: flex lays children in a row, justify-content: space-between pushes the logo left and the controls right, and align-items: center centres them vertically.
51–58	<pre>.nav-brand { font-size: 1.3rem; font-weight: 700; color: var(--primary); display: flex; align-items: center; gap: 10px; }</pre>	.nav-brand (the logo): larger bold blue text; display: flex with gap: 10px puts the icon and name in a row with space between.
60–64	<pre>.nav-controls { display: flex; align-items: center; gap: 15px; }</pre>	.nav-controls : a flex row with 15px between the user chip and the buttons.
66–75	<pre>.role-switch { display: flex; align-items: center; gap: 8px; background: #f1f5f9; padding: 6px 12px; border-radius: var(--radius); font-weight: 600; font-size: 0.9rem; }</pre>	.role-switch : styled the old Active Role dropdown (grey pill with padding and rounded corners). The HTML no longer uses it after the login update, so this rule can be deleted.
77–84	<pre>.role-switch select { border: none; background: transparent; font-weight: 600; color: var(--primary); cursor: pointer; outline: none; }</pre>	.role-switch select : removed the dropdown border and made its text blue and bold. Also unused now.
86	<pre>/* Hero & Search */</pre>	A comment marking the Hero and Search section.
87–93	<pre>.hero-banner { background: linear-gradient(135deg, #1e293b, #0f172a); color: white; padding: 35px 25px; border-radius: var(--radius); margin-bottom: 25px; }</pre>	.hero-banner : a diagonal gradient (linear-gradient(135deg, dark, darker)), white text, padding, rounded corners and space below.

Lines	Code	What it does
95	<code>.hero-banner h2 { font-size: 1.8rem; margin-bottom: 8px; }</code>	The h2 inside the hero: bigger font size and a small gap below.
96	<code>.hero-banner p { color: #94a3b8; margin-bottom: 20px; }</code>	The p inside the hero: soft grey-blue text and a gap below.
98–105	<code>.search-box { display: flex; gap: 10px; flex-wrap: wrap; background: white; padding: 8px; border-radius: var(--radius); }</code>	.search-box : a white flex row with a gap. flex-wrap: wrap lets items move to the next line on narrow screens.
107–114	<code>.input-icon-group { flex: 2; min-width: 200px; display: flex; align-items: center; padding-left: 12px; color: var(--text-muted); }</code>	.input-icon-group (search field with icon): flex: 2 makes it twice as wide as the dropdowns, min-width stops it getting too small, and the icon colour is muted grey.
116–122	<code>.input-icon-group input { width: 100%; border: none; padding: 10px; outline: none; font-size: 0.95rem; }</code>	The search input : full width, no border, padding, no focus outline and a readable font size.
124–131	<code>.search-box select { flex: 1; min-width: 130px; border: 1px solid var(--border); border-radius: calc(var(--radius) - 2px); padding: 8px 12px; outline: none; }</code>	The dropdowns inside the search box: flex: 1 shares space equally, a light border, slightly smaller rounded corners (calc(var(--radius) - 2px)) and padding.
133	<code>/* View Header & Tabs */</code>	A comment marking the View Header and Tabs section.
134–141	<code>.view-header { display: flex; justify-content: space-between; align-items: center; margin-bottom: 20px; flex-wrap: wrap; gap: 10px; }</code>	.view-header : a flex row with the count on the left and the tabs on the right, 20px below, and flex-wrap: wrap for small screens.
143	<code>.results-count { font-weight: 600; color: var(--text-muted); }</code>	.results-count : bold muted text for "Showing X of Y jobs".
145	<code>.tab-filters { display: flex; gap: 8px; }</code>	.tab-filters : puts the tab buttons in a row with 8px gaps.

Lines	Code	What it does
147–156	<pre>.tab-btn { background: transparent; border: 1px solid var(--border); padding: 8px 16px; border-radius: 20px; cursor: pointer; font-weight: 600; color: var(--text-muted); transition: all 0.2s; }</pre>	.tab-btn : pill-shaped buttons (border-radius: 20px), transparent with a light border, bold muted text, pointer cursor, and transition: all 0.2s so colour changes animate smoothly.
158–162	<pre>.tab-btn.active, .tab-btn:hover { background: var(--primary); color: white; border-color: var(--primary); }</pre>	A selector list (comma): the style applies to the active tab and to any tab under the mouse (:hover). .tab-btn.active means an element having both classes. Background turns blue and the text white.
164	<pre>/* Job Cards Grid */</pre>	A comment marking the Job Cards Grid section.
165–169	<pre>.cards-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(320px, 1fr)); gap: 20px; }</pre>	.cards-grid : display: grid with repeat(auto-fill, minmax(320px, 1fr)) makes as many columns as fit, each at least 320px wide, sharing leftover space. This is why the layout adapts to any screen. gap: 20px spaces the cards.
171–182	<pre>.job-card { background: var(--card-bg); border: 1px solid var(--border); border-radius: var(--radius); padding: 20px; box-shadow: var(--shadow); display: flex; flex-direction: column; justify-content: space-between; position: relative; transition: transform 0.2s, box-shadow 0.2s; }</pre>	.job-card : white box with border, rounded corners, padding and shadow. flex-direction: column stacks the content and justify-content: space-between pushes the footer to the bottom. position: relative anchors child elements, and transition animates the hover effect.
184–187	<pre>.job-card:hover { transform: translateY(-3px); box-shadow: var(--shadow-lg); }</pre>	.job-card:hover : when the mouse is over a card it moves up 3px (translateY(-3px)) and gets a larger shadow.
189–193	<pre>.job-card-header { display: flex; justify-content: space-between; align-items: flex-start; }</pre>	.job-card-header : flex row with the title on the left and the bookmark on the right, aligned to the top (flex-start).
195	<pre>.job-card h3 { font-size: 1.1rem; color: var(--text-dark); margin-bottom: 4px; }</pre>	The job title (h3) inside a card: font size, dark colour, small gap below.
196	<pre>.company-name { color: var(--primary); font-weight: 600; font-size: 0.9rem; margin-bottom: 4px; }</pre>	.company-name : blue, semi-bold, smaller text.

Lines	Code	What it does
197	<pre>.job-location { color: var(--text-muted); font-size: 0.82rem; margin-bottom: 12px; }</pre>	.job-location : small muted text for the city line.
198	<pre>.job-location i { margin-right: 4px; }</pre>	The location icon gets 4px on its right so it does not touch the text.
200–207	<pre>.bookmark-btn { background: transparent; border: none; font-size: 1.2rem; color: #cbd5e1; cursor: pointer; transition: color 0.2s; }</pre>	.bookmark-btn : removes the default button look (transparent, no border), pale grey icon, bigger size, pointer cursor and a colour transition.
209	<pre>.bookmark-btn.active { color: var(--accent-yellow); }</pre>	.bookmark-btn.active : a saved job's bookmark turns yellow.
211	<pre>.badges-row { display: flex; gap: 6px; flex-wrap: wrap; margin-bottom: 12px; }</pre>	.badges-row : category and type badges in a row with small gaps; flex-wrap lets them wrap.
213–218	<pre>.badge { font-size: 0.75rem; padding: 3px 8px; border-radius: 4px; font-weight: 600; }</pre>	.badge : small text with tight padding, slightly rounded corners, bold.
220–222	<pre>.badge-blue { background: #dbeafe; color: #1e40af; } .badge-green { background: #d1fae5; color: #0065f4; } .badge-purple { background: #f3e8ff; color: #6b21a8; }</pre>	Three badge colour themes: blue (category), green (job type) and purple ("Applied").
224–233	<pre>.job-desc { font-size: 0.88rem; color: var(--text-muted); margin-bottom: 15px; display: -webkit-box; -webkit-line-clamp: 3; line-clamp: 3; -webkit-box-orient: vertical; overflow: hidden; }</pre>	.job-desc : muted text. The -webkit-box , line-clamp: 3 , box-orient: vertical and overflow: hidden combination cuts the description after 3 lines and shows an ellipsis (...).
235–242	<pre>.card-footer { display: flex; justify-content: space-between; align-items: center; border-top: 1px solid #f1f5f9; padding-top: 12px; margin-top: auto; }</pre>	.card-footer : a flex row with salary left and the Apply button right, a thin top line, and margin-top: auto which pushes the footer to the bottom of the card.
244	<pre>.salary-tag { font-weight: 700; font-size: 0.95rem; color: var(--text-dark); }</pre>	.salary-tag : bold salary text.
246	<pre>/* Employer Dashboard */</pre>	A comment marking the Employer Dashboard section.

Lines	Code	What it does
247–252	<pre>.dashboard-header { display: flex; justify-content: space-between; align-items: center; margin-bottom: 20px; }</pre>	.dashboard-header : title on the left and button on the right.
254–259	<pre>.stats-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 15px; margin-bottom: 25px; }</pre>	.stats-grid : grid with <code>repeat(auto-fit, minmax(200px, 1fr))</code> so the stat cards share the width and wrap on small screens.
261–269	<pre>.stat-card { background: var(--card-bg); padding: 20px; border-radius: var(--radius); border: 1px solid var(--border); display: flex; align-items: center; gap: 15px; }</pre>	.stat-card : white box with border, flex row, centred, 15px between icon and text.
271	<pre>.stat-icon { font-size: 2rem; }</pre>	.stat-icon : large icon size.
272–273	<pre>.text-blue { color: var(--primary); } .text-green { color: var(--accent-green); }</pre>	Helper classes that colour an icon blue or green using the variables.
275	<pre>/* Table Styling */</pre>	A comment marking the Table Styling section.
276–282	<pre>.table-card { background: var(--card-bg); border-radius: var(--radius); border: 1px solid var(--border); padding: 20px; box-shadow: var(--shadow); }</pre>	.table-card : white box with border, padding and shadow around the table.
284	<pre>.table-wrapper { overflow-x: auto; margin-top: 15px; }</pre>	.table-wrapper : <code>overflow-x: auto</code> adds a horizontal scrollbar if the table is wider than the screen.
286–290	<pre>.data-table { width: 100%; border-collapse: collapse; text-align: left; }</pre>	.data-table : full width. <code>border-collapse: collapse</code> merges cell borders into single lines. Text is left aligned.
292–296	<pre>.data-table th, .data-table td { padding: 12px 15px; border-bottom: 1px solid var(--border); font-size: 0.9rem; }</pre>	Header and data cells: padding, a thin bottom border and a smaller font.

Lines	Code	What it does
298	<code>.data-table th { background: #f8fafc; font-weight: 600; }</code>	Header cells get a light grey background and semi-bold text.
300	<code>/* Buttons */</code>	A comment marking the Buttons section.
301–312	<code>.btn { padding: 8px 16px; border-radius: var(--radius); border: none; font-weight: 600; cursor: pointer; display: inline-flex; align-items: center; gap: 6px; font-size: 0.9rem; transition: background 0.2s; }</code>	<code>.btn</code> : the base style for all buttons: padding, rounded corners, no border, bold text, pointer cursor. <code>display: inline-flex</code> with <code>gap</code> aligns an icon and text; <code>transition</code> smooths hover colour changes.
314–319	<code>.btn-primary { background: var(--primary); color: white; } .btn-primary:hover { background: var(--primary-hover); } .btn-secondary { background: #e2e8f0; color: var(--text-dark); } .btn-secondary:hover { background: #cbd5e1; } .btn-danger { background: var(--accent-red); color: white; } .btn-sm { padding: 5px 10px; font-size: 0.8rem; }</code>	Button variants: primary (blue, darker on hover), secondary (grey, darker on hover), danger (red) and <code>.btn-sm</code> (smaller padding and font).
321	<code>/* Modals */</code>	A comment marking the Modals section.
322–331	<code>.modal-overlay { position: fixed; top:0; left:0; width: 100%; height: 100%; background: rgba(15, 23, 42, 0.6); display: flex; justify-content: center; align-items: center; z-index: 1000; padding: 20px; }</code>	<code>.modal-overlay</code> : <code>position: fixed</code> with <code>top/left 0</code> and <code>width/height 100%</code> covers the entire screen. A semi-transparent dark background (<code>rgba</code>) dims the page, flex centring puts the dialog in the middle, and <code>z-index: 1000</code> places it above the page.
333–342	<code>.modal-content { background: var(--card-bg); width: 100%; max-width: 550px; border-radius: var(--radius); padding: 25px; box-shadow: var(--shadow-lg); max-height: 90vh; overflow-y: auto; }</code>	<code>.modal-content</code> : the white dialog. <code>width: 100%</code> with <code>max-width: 550px</code> makes it responsive. <code>max-height: 90vh</code> (90% of screen height) plus <code>overflow-y: auto</code> makes long forms scroll inside the dialog.
344	<code>.modal-lg { max-width: 750px; }</code>	<code>.modal-lg</code> : a wider dialog (750px) for the applicants list.

Lines	Code	What it does
346–353	<pre>.modal-header { display: flex; justify-content: space-between; align-items: center; margin-bottom: 15px; border-bottom: 1px solid var(--border); padding-bottom: 10px; }</pre>	.modal-header : flex row with title and close button, a line underneath and spacing.
355–361	<pre>.close-btn { background: none; border: none; font-size: 1.5rem; cursor: pointer; color: var(--text-muted); }</pre>	.close-btn : the × button with no background or border, a larger size and a muted colour.
363	<pre>.form-group { margin-bottom: 15px; }</pre>	.form-group : spacing below each field.
364	<pre>.form-group label { display: block; font-weight: 600; font-size: 0.85rem; margin-bottom: 5px; }</pre>	The label inside a field group: display: block puts it on its own line; small bold text.
365–372	<pre>.form-group input, .form-group select, .form-group textarea { width: 100%; padding: 9px 12px; border: 1px solid var(--border); border-radius: var(--radius); outline: none; font-size: 0.9rem; }</pre>	The inputs, selects and textareas: full width, padding, a light border, rounded corners, no focus outline and a readable size. The three selectors are separated by commas.
374–375	<pre>.form-row { display: flex; gap: 15px; } .form-row .form-group { flex: 1; }</pre>	.form-row places fields side by side (flex with a gap). .form-row .form-group (a group inside a row) gets flex: 1 so the fields share the width equally.
377	<pre>.help-text { font-size: 0.75rem; color: var(--text-muted); margin-top: 3px; display: block; }</pre>	.help-text : small muted helper text shown as a block.
378	<pre>.modal-footer { display: flex; justify-content: flex-end; gap: 10px; margin-top: 20px; }</pre>	.modal-footer : buttons aligned to the right (flex-end) with a gap and space above.
380	<pre>/* Applicant Item Cards in Modal */</pre>	A comment marking the Applicant Cards section.
381–387	<pre>.applicant-card { border: 1px solid var(--border); padding: 15px; border-radius: var(--radius); margin-bottom: 10px; background: #f8fafc; }</pre>	.applicant-card : bordered box with a light grey background for each applicant.

Lines	Code	What it does
389–394	<pre>.applicant-card-header { display: flex; justify-content: space-between; align-items: center; margin-bottom: 8px; }</pre>	<code>.applicant-card-header</code> : flex row, name on the left and status dropdown on the right.
396–401	<pre>.status-select { padding: 4px 8px; border-radius: 4px; font-weight: 600; font-size: 0.8rem; }</pre>	<code>.status-select</code> : padding, rounded corners and a small bold font for the status dropdown.
403	<pre>/* Toast Notifications */</pre>	A comment marking the Toast Notifications section.
404–412	<pre>.toast-container { position: fixed; bottom: 20px; right: 20px; z-index: 2000; display: flex; flex-direction: column; gap: 10px; }</pre>	<code>.toast-container</code> : fixed to the bottom-right corner (<code>bottom: 20px; right: 20px</code>). <code>z-index: 2000</code> keeps it above everything. <code>flex-direction: column</code> stacks several toasts with a gap.
414–425	<pre>.toast { background: #1e293b; color: white; padding: 12px 20px; border-radius: var(--radius); box-shadow: var(--shadow-lg); font-size: 0.9rem; display: flex; align-items: center; gap: 10px; animation: slideIn 0.3s forwards; }</pre>	<code>.toast</code> : a dark rounded popup with white text and a shadow. <code>animation: slideIn 0.3s forwards</code> plays the slide-in animation once and keeps the final state.
427–430	<pre>@keyframes slideIn { from { transform: translateX(100%); opacity: 0; } to { transform: translateX(0); opacity: 1; } }</pre>	<code>@keyframes slideIn</code> defines the animation: it starts moved fully to the right (<code>translateX(100%)</code>) and invisible (<code>opacity: 0</code>), then ends in place and fully visible.
432–434	<pre>/* ===== Auth screen (login / sign up) ===== */</pre>	A multi-line comment marking the Auth screen section (added with login/sign-up).

Lines	Code	What it does
435–443	<pre>.auth-screen { position: fixed; inset: 0; z-index: 1500; display: grid; grid-template-columns: 1fr 1fr; background: var(--bg-main); overflow-y: auto; }</pre>	.auth-screen : position: fixed with inset: 0 (shorthand for top, right, bottom and left all 0) covers the full screen. z-index: 1500 puts it above modals (1000) but below toasts (2000). display: grid with grid-template-columns: 1fr 1fr makes two equal columns. overflow-y: auto allows scrolling on short screens.
445–453	<pre>.auth-aside { background: linear-gradient(135deg, #1e293b, #0f172a); color: white; padding: 48px 56px; display: flex; flex-direction: column; justify-content: center; gap: 18px; }</pre>	.auth-aside : the dark left column with a gradient, white text and large padding. flex-direction: column with justify-content: center centres its content vertically.
455	<pre>.auth-brand { color: white; margin-bottom: 24px; }</pre>	.auth-brand : white logo and space below.
456	<pre>.auth-aside h1 { font-size: 2.2rem; line-height: 1.2; max-width: 16ch; }</pre>	The headline: large size, tight line-height , and max-width: 16ch (about 16 characters) to control the line breaks.
457	<pre>.auth-aside p { color: #94a3b8; max-width: 42ch; }</pre>	The description: muted colour and a max-width of about 42 characters for easy reading.
459	<pre>.auth-points { list-style: none; display: grid; gap: 10px; margin-top: 8px; }</pre>	.auth-points : removes the bullet dots (list-style: none) and uses a grid with gaps.
460	<pre>.auth-points li { display: flex; align-items: center; gap: 10px; color: #cbd5e1; font-size: 0.95rem; }</pre>	Each list item: flex row with the icon and text aligned.
461	<pre>.auth-points i { color: var(--accent-green); }</pre>	The tick icons are green.
463	<pre>.auth-panel { display: flex; align-items: center; justify-content: center; padding: 32px 20px; }</pre>	.auth-panel : centres the card both ways using flex and adds padding.
465–473	<pre>.auth-card { width: 100%; max-width: 420px; background: var(--card-bg); border: 1px solid var(--border); border-radius: var(--radius); box-shadow: var(--shadow-lg); padding: 28px; }</pre>	.auth-card : white card, max width 420px, border, rounded corners, a big shadow and padding.

Lines	Code	What it does
475–482	<pre>.auth-tabs { display: grid; grid-template-columns: 1fr 1fr; background: #f1f5f9; border-radius: var(--radius); padding: 4px; margin-bottom: 22px; }</pre>	.auth-tabs : a two-column grid on a light grey background, like a segmented control, with 4px padding.
484–493	<pre>.auth-tab { background: transparent; border: none; padding: 9px; border-radius: calc(var(--radius) - 2px); font-weight: 600; font-size: 0.9rem; color: var(--text-muted); cursor: pointer; }</pre>	.auth-tab : transparent buttons without borders, bold muted text and a pointer cursor.
495	<pre>.auth-tab.active { background: white; color: var(--primary); box-shadow: var(--shadow); }</pre>	.auth-tab.active : the selected tab gets a white background, blue text and a small shadow.
497	<pre>.auth-title { font-size: 1.35rem; margin-bottom: 4px; }</pre>	.auth-title : heading size and small space below.
498	<pre>.auth-sub { color: var(--text-muted); font-size: 0.9rem; margin-bottom: 20px; }</pre>	.auth-sub : muted helper text.
500–508	<pre>.auth-error { background: #fef2f2; border: 1px solid #fecaca; color: #b91c1c; padding: 10px 12px; border-radius: var(--radius); font-size: 0.85rem; margin-bottom: 16px; }</pre>	.auth-error : light red background, red border and text, padding and rounded corners for error messages.
510	<pre>.password-field { position: relative; }</pre>	.password-field : position: relative so the eye button can be positioned inside it.
511	<pre>.password-field input { padding-right: 42px; }</pre>	Extra right padding on the password input so the text never goes under the eye icon.
512–522	<pre>.toggle-pw { position: absolute; right: 4px; top: 50%; transform: translateY(-50%); background: none; border: none; color: var(--text-muted); padding: 8px 10px; cursor: pointer; }</pre>	.toggle-pw : position: absolute puts the eye button inside the field at the right. top: 50% with transform: translateY(-50%) is the standard trick to centre it vertically. It has no background or border.

Lines	Code	What it does
524	<code>.role-picker { margin-bottom: 15px; }</code>	<code>.role-picker</code> : space below the whole role section.
525	<code>.role-legend { display: block; font-weight: 600; font-size: 0.85rem; margin-bottom: 5px; }</code>	<code>.role-legend</code> : label-style heading "I want to" shown as a block.
526	<code>.role-options { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; }</code>	<code>.role-options</code> : a two-column grid so the two choices are equal and side by side.
527	<code>.role-option { position: relative; cursor: pointer; }</code>	<code>.role-option</code> : <code>position: relative</code> so the hidden radio can be placed over it; pointer cursor.
528	<code>.role-option input { position: absolute; opacity: 0; inset: 0; width: 100%; height: 100%; cursor: pointer; }</code>	The real radio input is made invisible (<code>opacity: 0</code>) but stretched across the whole card (<code>position: absolute; inset: 0; width/height 100%</code>), so clicking anywhere on the card selects it and keyboard use still works.
529–540	<code>.role-option span { display: flex; align-items: center; justify-content: center; gap: 8px; padding: 10px; border: 1px solid var(--border); border-radius: var(--radius); font-size: 0.88rem; font-weight: 600; color: var(--text-muted); }</code>	<code>.role-option span</code> : the visible card: flex centred icon and text, padding, border, rounded corners and muted bold text.
541	<code>.role-option input:checked + span { border-color: var(--primary); background: #eff6ff; color: var(--primary); }</code>	<code>.role-option input:checked + span</code> : the <code>+</code> is the adjacent sibling selector. It styles the span right after a checked radio with a blue border, light blue background and blue text.
542	<code>.role-option input:focus-visible + span { outline: 2px solid var(--primary); outline-offset: 2px; }</code>	<code>:focus-visible</code> shows a blue outline only when the card is reached by keyboard, which helps accessibility.
544	<code>.auth-submit { width: 100%; justify-content: center; padding: 11px; margin-top: 4px; }</code>	<code>.auth-submit</code> : full-width button with centred content.
545	<code>.auth-switch { text-align: center; font-size: 0.85rem; color: var(--text-muted); margin-top: 16px; }</code>	<code>.auth-switch</code> : centred small muted text with space above.
546	<code>.link-btn { background: none; border: none; color: var(--primary); font-weight: 600; cursor: pointer; font-size: inherit; }</code>	<code>.link-btn</code> : a button that looks like a link (no background or border, blue bold text).
547	<code>.link-btn:hover { text-decoration: underline; }</code>	Underlines the link-button on hover.
549–550	<code>.auth-tab:focus-visible, .link-btn:focus-visible, .toggle-pw:focus-visible, .form-group input:focus-visible { outline: 2px solid var(--primary); outline-offset: 1px; }</code>	Keyboard-focus outlines (blue, 2px) for tabs, link buttons, the eye button and form inputs, so keyboard users can see where they are.
552	<code>/* Logged-in user chip in navbar */</code>	A comment marking the user chip section.
553	<code>.user-chip { display: flex; align-items: center; gap: 10px; }</code>	<code>.user-chip</code> : avatar and name in a row with 10px gap.

Lines	Code	What it does
554–561	<pre> .avatar { width: 34px; height: 34px; border-radius: 50%; background: var(--primary); color: white; display: flex; align-items: center; justify-content: center; font-weight: 700; } </pre>	.avatar : a 34px circle (border-radius: 50%) with a blue background, white bold letter, centred with flex.
562	<pre> .user-meta { display: flex; flex-direction: column; line-height: 1.2; font-size: 0.85rem; } </pre>	.user-meta : stacks name above role (flex-direction: column) with tight line height and small text.
563	<pre> .user-meta small { color: var(--text-muted); } </pre>	The role label (small) uses the muted colour.
565–573	<pre> @media (max-width: 820px) { .auth-screen { grid-template-columns: 1fr; } .auth-aside { padding: 28px 24px; gap: 10px; } .auth-brand { margin-bottom: 8px; } .auth-aside h1 { font-size: 1.5rem; } .auth-points { display: none; } .navbar { flex-direction: column; gap: 10px; } .nav-controls { flex-wrap: wrap; justify-content: center; } } </pre>	@media (max-width: 820px) is a media query: the rules inside apply only on screens 820px wide or narrower (phones and small tablets). The auth screen becomes one column, the left panel gets smaller padding and headline, the bullet list is hidden, the navbar stacks vertically and the controls wrap and centre.

Part 3: script.js (the behaviour)

The JavaScript file holds the data, the logic and the screen updates. It is organised in seven numbered sections, from the demo data to the login system.

Lines	Code	What it does
Section 1: Seed data and global state		
1–3	// ===== // 1. GLOBAL STATE & LOCAL STORAGE SEED DATA // =====	A banner comment for section 1. Comments start with <code>//</code> and are ignored by the browser.
5	const SEED_JOBS = [	<code>const SEED_JOBS = [</code> starts an array (a list in square brackets) of job objects. These are the demo jobs shown the first time. <code>const</code> means the variable cannot be reassigned.
6–15	{ id: 101, title: "Senior Full Stack Engineer", company: "Apex Tech Labs", location: "Bengaluru, Karnataka", category: "Engineering", type: "Full-time", salary: "₹28 - 40 LPA", description: "Lead our core product team using React, Node.js and PostgreSQL. Experience building scalable systems for Indian consumer apps (UPI, KYC, regional-language support) is a plus." },	One job is an object <code>{ ... }</code> made of <code>key: value</code> pairs. <code>id</code> is a unique number used to find the job, <code>title</code> , <code>company</code> and <code>location</code> are text, <code>category</code> and <code>type</code> are used by the filters, <code>salary</code> is text in ₹ LPA (lakhs per annum), and <code>description</code> is the paragraph on the card. Every job in the list has the same fields.
16–125	... (same structure, see the file)	Eleven more job objects with ids 102 to 112, built exactly like the first: Product Designer, Growth Marketing Manager, Data Analyst, Software Engineering Intern, Machine Learning Engineer, Associate Product Manager, Vernacular Content Executive, Embedded/IoT Engineer, Cloud & DevOps Engineer, UI/UX Design Intern and Cybersecurity Analyst.
126	];	<code>];</code> closes the array.
128	let currentUser = null; // set after login (see section 7)	<code>let currentUser = null;</code> remembers who is logged in. <code>let</code> allows the value to change later. <code>null</code> means "nobody yet". It is set after login.
130–132	// Bump this number whenever SEED_JOBS changes. Old cached jobs (e.g. the \$ ones) // are replaced automatically; saved jobs and applications are kept. const SEED_VERSION = '2';	Comments explain the next lines, then <code>const SEED_VERSION = '2';</code> stores a version label for the seed data.
133–136	if (localStorage.getItem('hp_seedVersion') !== SEED_VERSION) { localStorage.removeItem('hp_jobs'); localStorage.setItem('hp_seedVersion', SEED_VERSION); }	<code>localStorage</code> is small storage inside the browser that survives refreshes. If the saved version differs from <code>SEED_VERSION</code> (<code>!==</code> means "not equal"), the old cached jobs are removed and the new version is saved. This is why updating the seed jobs and bumping the number refreshes old browsers.
138–139	// Load application state from localStorage or initialize defaults let appState = {	A comment and the start of <code>let appState = {</code> , one object that holds the whole app state.

Lines	Code	What it does
140–141	<pre> activeRole: 'seeker', // 'seeker' 'employer' activeSeekerTab: 'all', // 'all' 'saved' 'applied' </pre>	<code>activeRole</code> is 'seeker' or 'employer'; <code>activeSeekerTab</code> is 'all', 'saved' or 'applied'. The text after <code>//</code> is a comment listing the allowed values.
142	<pre> savedJobIds: [], // loaded per user after login </pre>	<code>savedJobIds: []</code> is an empty list of bookmarked job ids; it is loaded for the current user after login.
143	<pre> jobs: JSON.parse(localStorage.getItem('hp_jobs')) SEED_JOBS, </pre>	<code>JSON.parse(localStorage.getItem('hp_jobs'))</code> reads saved text and turns it into a real array. <code> SEED_JOBS</code> means "or": if nothing is saved, use the demo jobs.
144–145	<pre> applications: JSON.parse(localStorage.getItem('hp_applications')) [] }; </pre>	Applications work the same way, with an empty list <code>[]</code> as the fallback. <code>};</code> ends the object.
147–148	<pre> // Initialize persistent storage if empty function syncStorage() { </pre>	A comment and the start of <code>function syncStorage()</code> , which saves the current state to the browser.
149	<pre> localStorage.setItem('hp_seedVersion', SEED_VERSION); </pre>	Saves the seed version label.
150	<pre> localStorage.setItem('hp_jobs', JSON.stringify(appState.jobs)); </pre>	<code>JSON.stringify</code> turns the jobs array into text, because <code>localStorage</code> can only store strings.
151	<pre> if (currentUser) localStorage.setItem('hp_savedJobs_' + currentUser.id, JSON.stringify(appState.savedJobIds)); </pre>	Saves bookmarks per user using a key like <code>hp_savedJobs_u123</code> . The <code>if (currentUser)</code> check means it only runs when someone is logged in. <code>+</code> joins text together.
152–153	<pre> localStorage.setItem('hp_applications', JSON.stringify(appState.applications)); } </pre>	Saves applications, then <code>}</code> closes the function.

Section 2: Core rendering

155–157	<pre> // ===== // 2. CORE RENDERING ENGINE // ===== </pre>	A banner comment for section 2.
159	<pre> document.addEventListener('DOMContentLoaded', () => { </pre>	<code>document.addEventListener('DOMContentLoaded', () => {</code> waits until the HTML has fully loaded, then runs the function. <code>() => {}</code> is an arrow function (a short way to write a function).
160–162	<pre> if (!localStorage.getItem('hp_jobs')) { syncStorage(); } </pre>	If no jobs are saved yet (first visit), <code>!</code> means "not", so <code>syncStorage()</code> stores the starting data.
163–164	<pre> restoreSession(); }); </pre>	<code>restoreSession()</code> checks if someone is already logged in and shows either the app or the login screen. <code>});</code> closes the listener.
166–167	<pre> function render() { if (appState.activeRole === 'seeker') { </pre>	<code>function render()</code> redraws the right screen. <code>if (appState.activeRole === 'seeker')</code> uses <code>===</code> to check for an exact match.
168–170	<pre> document.getElementById('seekerSection').classList.remove('hidden'); document.getElementById('employerSection').classList.add('hidden'); renderSeekerGrid(); </pre>	For seekers: show the seeker section (<code>classList.remove('hidden')</code>), hide the employer section (<code>classList.add('hidden')</code>), then draw the job cards with <code>renderSeekerGrid()</code> .

Lines	Code	What it does
171–175	<pre> } else { document.getElementById('seekerSection').classList.add('hidden'); document.getElementById('employerSection').classList.remove('hidden'); renderEmployerDashboard(); } </pre>	<code>else</code> is for employers: the opposite sections are shown and <code>renderEmployerDashboard()</code> draws the table.
176–177	<pre> updateCounters(); } </pre>	<code>updateCounters()</code> refreshes the numbers on the tabs, then <code>}</code> closes the function.
Section 3: Job seeker module and filters		
179–181	<pre> // ===== // 3. JOB SEEKER MODULE & FILTER ENGINE // ===== </pre>	A banner comment for section 3.
183–185	<pre> function handleSearchAndFilter() { renderSeekerGrid(); } </pre>	<code>handleSearchAndFilter()</code> is called by the search box and dropdowns (in the HTML) and simply redraws the grid, which re-reads their current values.
187–188	<pre> function switchSeekerTab(tabName) { appState.activeSeekerTab = tabName; } </pre>	<code>switchSeekerTab(tabName)</code> receives <code>'all'</code> , <code>'saved'</code> or <code>'applied'</code> and stores it in the state.
189	<pre> document.querySelectorAll('.tab-btn').forEach(btn => btn.classList.remove('active')); </pre>	<code>querySelectorAll('.tab-btn')</code> finds every tab button; <code>.forEach(btn => ...)</code> loops over them and removes the <code>active</code> class from each.
191–193	<pre> if (tabName === 'all') document.getElementById('tabAllJobs').classList.add('active'); if (tabName === 'saved') document.getElementById('tabSavedJobs').classList.add('active'); if (tabName === 'applied') document.getElementById('tabMyApps').classList.add('active'); </pre>	Three <code>if</code> lines add the <code>active</code> class to the tab that was clicked.
195–196	<pre> renderSeekerGrid(); } </pre>	Redraws the grid, then <code>}</code> ends the function.
198–202	<pre> function renderSeekerGrid() { const container = document.getElementById('jobGrid'); const searchVal = document.getElementById('searchInput').value.toLowerCase().trim(); const categoryVal = document.getElementById('categoryFilter').value; const typeVal = document.getElementById('typeFilter').value; } </pre>	<code>renderSeekerGrid()</code> begins. It finds the grid container, and reads the search text (<code>.toLowerCase()</code> makes it case-insensitive, <code>.trim()</code> removes extra spaces), the chosen category and the chosen type.
204–205	<pre> // Filter pipeline let filtered = appState.jobs.filter(job => { </pre>	A comment and <code>let filtered = appState.jobs.filter(job => {</code> . <code>.filter()</code> keeps only the jobs for which the function returns <code>true</code> .
206–209	<pre> const matchesSearch = job.title.toLowerCase().includes(searchVal) job.company.toLowerCase().includes(searchVal) (job.location '').toLowerCase().includes(searchVal) job.description.toLowerCase().includes(searchVal) }); </pre>	<code>matchesSearch</code> is true if the typed text appears (<code>.includes</code>) in the title, company, location or description. <code> </code> means OR. <code>(job.location '')</code> protects jobs that have no location so the code does not crash.

Lines	Code	What it does
210–211	<pre>const matchesCategory = categoryVal === 'all' job.category === categoryVal; const matchesType = typeVal === 'all' job.type === typeVal;</pre>	<code>matchesCategory</code> is true if the filter is 'all' or the job's category equals the chosen one. <code>matchesType</code> works the same way for the type.
213–214	<pre>return matchesSearch && matchesCategory && matchesType; });</pre>	<code>return matchesSearch && matchesCategory && matchesType;</code> uses && (AND), so a job must pass all three checks to stay in the list. <code>});</code> ends the filter.
216–218	<pre>// Filter based on active tab if (appState.activeSeekerTab === 'saved') { filtered = filtered.filter(j => appState.savedJobIds.includes(j.id));</pre>	If the Bookmarked tab is active, keep only jobs whose id is in <code>savedJobIds</code> (<code>includes</code>).
219–222	<pre>} else if (appState.activeSeekerTab === 'applied') { const appliedJobIds = myApplications().map(a => a.jobId); filtered = filtered.filter(j => appliedJobIds.includes(j.id)); }</pre>	<code>else if</code> the My Applications tab is active: <code>myApplications().map(a => a.jobId)</code> builds a list of job ids you applied to, and the jobs are filtered to those ids.
224	<pre>document.getElementById('jobCountBadge').innerText = `Showing \${filtered.length} of \${appState.jobs.length} jobs`;</pre>	Updates the "Showing X of Y jobs" text. The back-tick quotes make a template literal, where <code>\${...}</code> inserts a value into the text.
226–234	<pre>if (filtered.length === 0) { container.innerHTML = `     <div style="grid-column: 1/-1; text-align: center; padding: 40px; color: var(--text-muted);">       <i class="fa-solid fa-folder-open" style="font-size: 2.5rem; margin-bottom: 10px;"></i>       <p>No jobs found matching your criteria.</p>     </div>   `; return; }</pre>	If nothing matches, an "empty state" message with an icon is placed in the container with <code>innerHTML</code> , then <code>return</code> stops the function early. <code>grid-column: 1/-1</code> makes the message span the full grid width.
236–239	<pre>container.innerHTML = filtered.map(job => { const isSaved = appState.savedJobIds.includes(job.id); const hasApplied = myApplications().some(a => a.jobId === job.id);</pre>	<code>container.innerHTML = filtered.map(job => {</code> turns each job into a block of HTML. <code>isSaved</code> and <code>hasApplied</code> are booleans (true/false) used below.
240–248	<pre>return ` <div class="job-card"> <div> <div class="job-card-header"> <h3>\${escapeHTML(job.title)}</h3> <button class="bookmark-btn \${isSaved ? 'active' : ''}" onclick="toggleBookmark(\${job.id})" title="Save Job"> <i class="fa-\${isSaved ? 'solid' : 'regular'} fa-bookmark"></i> </button> </div> </div></pre>	Builds the start of one job card. <code>escapeHTML(...)</code> prevents text from being treated as HTML code. <code>\${isSaved ? 'active' : ''}</code> is the ternary operator: <code>condition ? valueIfTrue : valueIfFalse</code> . The bookmark button calls <code>toggleBookmark(id)</code> and the icon is solid or outline depending on <code>isSaved</code> .
249–250	<pre><div class="company-name">\${escapeHTML(job.company)}</div> \${job.location ? `<div class="job-location"><i class="fa-solid fa-location-dot"></i> \${escapeHTML(job.location)}</div>` : ''}</pre>	Shows the company name. The next line adds the location row only if the job has a location (otherwise it adds an empty string '').

Lines	Code	What it does
251–256	<pre> <div class="badges-row"> \${escapeHTML(job.category)} \${escapeHTML(job.type)} </div> <p class="job-desc">\${escapeHTML(job.description)}</p> </div> </pre>	The category and type badges, then the description paragraph, then the div closes.
257–263	<pre> <div class="card-footer"> \${escapeHTML(job.salary)} \${hasApplied ? `<span class="badge badge-purple"><i class="fa-solid fa-check"></i> Applied</span>` : `<button class="btn btn-primary btn-sm" onclick="openApplyModal(\${job.id})">Apply Now</button>`} </div> </pre>	The card footer shows the salary. If <code>hasApplied</code> is true it shows an "Applied" badge; otherwise an Apply Now button that calls <code>openApplyModal(id)</code> .
264–267	<pre> </div> `; }).join(''); } </pre>	Closes the card. <code>.join('')</code> glues the list of HTML strings into one big string, which becomes the grid's content. <code>}</code> ends the function.
269–279	<pre> function toggleBookmark(jobId) { if (appState.savedJobIds.includes(jobId)) { appState.savedJobIds = appState.savedJobIds.filter(id => id !== jobId); showToast("Job removed from bookmarks"); } else { appState.savedJobIds.push(jobId); showToast("Job saved to bookmarks!"); } syncStorage(); render(); } </pre>	<code>toggleBookmark(jobId)</code> : if the id is already saved, <code>.filter(id => id !== jobId)</code> removes it and shows a toast; otherwise <code>.push(jobId)</code> adds it. Then <code>syncStorage()</code> saves and <code>render()</code> redraws.
Section 4: Applications and file handling		
281–283	<pre> // ===== // 4. APPLICATION & BASE64 FILE HANDLING // ===== </pre>	A banner comment for section 4.
285–287	<pre> function openApplyModal(jobId) { const job = appState.jobs.find(j => j.id === jobId); if (!job) return; </pre>	<code>openApplyModal(jobId)</code> finds the job with <code>.find()</code> . <code>if (!job) return;</code> is a safety check that stops if the job does not exist.
289–290	<pre> document.getElementById('applyJobId').value = jobId; document.getElementById('applyModalTitle').innerText = `Apply for \${job.title} at \${job.company}`; </pre>	Stores the job id in the hidden input and sets the popup title using a template literal ("Apply for [title] at [company]").
291–294	<pre> if (currentUser) { document.getElementById('applicantName').value = currentUser.name; document.getElementById('applicantEmail').value = currentUser.email; } </pre>	If someone is logged in, the name and email fields are pre-filled from their account.

Lines	Code	What it does
295–296	<pre>openModal('applyModal'); }</pre>	Opens the popup, then <code>}</code> ends the function.
298–299	<pre>function handleApplySubmit(e) { e.preventDefault();</pre>	<code>handleApplySubmit(e)</code> runs on form submit. <code>e.preventDefault()</code> stops the browser from reloading the page.
300–303	<pre>const jobId = parseInt(document.getElementById('applyJobId').value); const name = document.getElementById('applicantName').value; const email = document.getElementById('applicantEmail').value.trim().toLowerCase(); const coverLetter = document.getElementById('coverLetter').value;</pre>	Reads the form: <code>parseInt</code> turns the job id from text into a number, then the name, email (trimmed and lower-cased) and cover letter.
304–305	<pre>const fileInput = document.getElementById('resumeUpload'); const file = fileInput.files[0];</pre>	Finds the file input and gets the chosen file with <code>.files[0]</code> (the first one).
307–311	<pre>// Prevent Duplicate Applications if (appState.applications.some(a => a.jobId === jobId && a.email === email)) { showToast("You have already applied for this job!", "danger"); return; }</pre>	Duplicate check: <code>.some()</code> returns true if any application already has this job id and email. If so, a toast is shown and <code>return</code> stops.
313–317	<pre>// File size validation (Max 2MB to prevent localStorage QuotaExceededError) if (file.size > 2 * 1024 * 1024) { showToast("File size exceeds 2MB limit for local storage!", "danger"); return; }</pre>	File size check: <code>2 * 1024 * 1024</code> is 2 MB in bytes. Larger files are rejected because <code>localStorage</code> has a size limit (about 5 MB).
319–321	<pre>const reader = new FileReader(); reader.onload = function(event) { const base64PDF = event.target.result; // Data URL string</pre>	<code>new FileReader()</code> reads the file. <code>reader.onload = function(event) {</code> runs after reading finishes. <code>event.target.result</code> is the file as a base64 data URL string.
323–332	<pre>const newApp = { id: Date.now(), jobId: jobId, applicantName: name, email: email, coverLetter: coverLetter, resumeBase64: base64PDF, status: 'Submitted', // 'Submitted' 'Shortlisted' 'Rejected' appliedAt: new Date().toLocaleDateString() };</pre>	Creates the application object: a unique <code>id</code> from <code>Date.now()</code> (milliseconds since 1970), the job id, name, email, cover letter, the resume text, a <code>status</code> of <code>'Submitted'</code> and today's date.
334–340	<pre>appState.applications.push(newApp); syncStorage(); closeModal('applyModal'); document.getElementById('applyForm').reset(); showToast("Application submitted successfully!"); render(); };</pre>	<code>.push()</code> adds it to the list, <code>syncStorage()</code> saves, then the popup closes, the form resets, a success toast shows and <code>render()</code> redraws.
342–343	<pre>reader.readAsDataURL(file); }</pre>	<code>reader.readAsDataURL(file)</code> starts the reading (it finishes later, which triggers <code>onload</code>). <code>}</code> ends the function.

Lines	Code	What it does
Section 5: Employer dashboard		
345–347	<pre>// ===== // 5. EMPLOYER DASHBOARD & CRUD // =====</pre>	A banner comment for section 5.
349–351	<pre>function renderEmployerDashboard() { document.getElementById('statTotalJobs').innerText = appState.jobs.length; document.getElementById('statTotalApps').innerText = appState.applications.length;</pre>	<code>renderEmployerDashboard()</code> writes the number of jobs and the number of applications into the two stat cards.
353–358	<pre>const tbody = document.getElementById('employerJobTableBody'); if (appState.jobs.length === 0) { tbody.innerHTML = `<tr><td colspan="5" style="text-align:center; padding: 20px;">No job postings available. Click "Post New Position" to create one.</td></tr>`; return; }</pre>	Finds the table body. If there are no jobs, it shows one row with <code>colspan="5"</code> (a cell spanning all five columns) and a message, then returns.
360–361	<pre>tbody.innerHTML = appState.jobs.map(job => { const jobApps = appState.applications.filter(a => a.jobId === job.id);</pre>	Builds one table row per job. <code>jobApps</code> is the list of applications for that job.
362–367	<pre> return ` <tr> <td>\${escapeHTML(job.title)}
<small class="text-muted">\${escapeHTML(job.company)}\${job.location ? ' · ' + escapeHTML(job.location) : ''}</small></td> <td>\${escapeHTML(job.category)}</td> <td>\${escapeHTML(job.type)}</td> <td>\${jobApps.length} candidates</td></pre>	The row: first cell shows title, company and location (<code>' · ' + ...</code> adds the dot and location), then the category badge, type and the applicant count.
368–379	<pre> <td> <button class="btn btn-secondary btn-sm" onclick="viewApplicants(\${job.id})"> <i class="fa-solid fa-eye"></i> Applicants (\${jobApps.length}) </button> <button class="btn btn-danger btn-sm" onclick="deleteJob(\${job.id})"> <i class="fa-solid fa-trash"></i> </button> </td> </tr> ` }).join(''); }</pre>	The actions cell: View Applicants button (calls <code>viewApplicants(id)</code>) and the trash button (calls <code>deleteJob(id)</code>). <code>.join('')</code> makes one HTML string.

Lines	Code	What it does
381–392	<pre>function handlePostJob(e) { e.preventDefault(); const newJob = { id: Date.now(), title: document.getElementById('jobTitle').value, company: document.getElementById('companyName').value, location: document.getElementById('jobLocation').value, category: document.getElementById('jobCategory').value, type: document.getElementById('jobType').value, salary: document.getElementById('jobSalary').value, description: document.getElementById('jobDescription').value }; }</pre>	<code>handlePostJob(e)</code> creates <code>newJob</code> , an object whose values come from the form fields; <code>id: Date.now()</code> gives it a unique id.
394–400	<pre>appState.jobs.unshift(newJob); syncStorage(); closeModal('postJobModal'); document.getElementById('postJobForm').reset(); showToast("New job posting published!"); render(); }</pre>	<code>.unshift()</code> adds the job at the start of the list so it appears first. Then it saves, closes the popup, resets the form, shows a toast and redraws.
402–409	<pre>function deleteJob(jobId) { if (confirm("Are you sure you want to delete this job posting? Associated applications will remain in records.)) { appState.jobs = appState.jobs.filter(j => j.id !== jobId); syncStorage(); showToast("Job post removed."); render(); } }</pre>	<code>deleteJob(jobId): confirm(...)</code> shows an OK/Cancel box. If OK, <code>.filter(j => j.id !== jobId)</code> keeps every job except this one. Then it saves, shows a toast and redraws.
411–416	<pre>function viewApplicants(jobId) { const job = appState.jobs.find(j => j.id === jobId); const apps = appState.applications.filter(a => a.jobId === jobId); const container = document.getElementById('applicantsListContainer'); document.getElementById('applicantsModalTitle').innerText = `Applicants for \${job ? job.title : 'Position'}`;</pre>	<code>viewApplicants(jobId)</code> finds the job and its applications, then sets the popup title. <code>job ? job.title : 'Position'</code> uses a fallback word if the job is missing.
418–420	<pre>if (apps.length === 0) { container.innerHTML = `<p style="text-align:center; color: var(--text-muted); padding: 20px;">No applications submitted yet for this position.</p>`; } else {</pre>	If there are no applications, a friendly message is shown.
421–427	<pre> container.innerHTML = apps.map(app => ` <div class="applicant-card"> <div class="applicant-card-header"> <div> \${escapeHTML(app.applicantName)} (\${escapeHTML(app.email)})
<small class="text-muted">Applied on: \${app.appliedAt}</small> </div></pre>	Otherwise <code>apps.map(...)</code> builds a card for each applicant: name, email and the applied date.

Lines	Code	What it does
428–435	<pre> <div> <select class="status-select" onchange="updateAppStatus(\${app.id}, this.value)"> <option value="Submitted" \${app.status === 'Submitted' ? 'selected' : ''}>Submitted</option> <option value="Shortlisted" \${app.status === 'Shortlisted' ? 'selected' : ''}>Shortlisted</option> <option value="Rejected" \${app.status === 'Rejected' ? 'selected' : ''}>Rejected</option> </select> </div> </div> </pre>	The status dropdown. <code>onchange</code> calls <code>updateAppStatus(id, this.value)</code> . For each option, <code>\${app.status === 'Submitted' ? 'selected' : ''}</code> pre-selects the current status.
436–439	<pre> <p style="font-size:0.85rem; margin-bottom: 8px;">\${escapeHTML(app.coverLetter 'No cover letter provided.')}</p> <i class="fa-solid fa-file-pdf"></i> Download Resume PDF </pre>	The cover letter in italics (or "No cover letter provided."), and the resume link. The <code>href</code> is the base64 data URL and <code>download="Resume_Name.pdf"</code> makes the browser download it.
440–445	<pre> </div> `).join(''); } openModal('viewApplicantsModal'); } </pre>	Closes the card, <code>.join('')</code> combines them, then <code>openModal('viewApplicantsModal')</code> shows the popup.
447–454	<pre> function updateAppStatus(appId, newStatus) { const app = appState.applications.find(a => a.id === appId); if (app) { app.status = newStatus; syncStorage(); showToast('Application status updated to \${newStatus}'); } } </pre>	<code>updateAppStatus(appId, newStatus)</code> finds the application, changes its <code>status</code> , saves and shows a toast.
Section 6: Utilities and modal controllers		
456–458	<pre> // ===== // 6. UTILITIES & MODAL CONTROLLERS // ===== </pre>	A banner comment for section 6.
460–463	<pre> function updateCounters() { document.getElementById('savedCount').innerText = appState.savedJobIds.length; document.getElementById('appliedCount').innerText = myApplications().length; } </pre>	<code>updateCounters()</code> writes the bookmarks count and your own applications count into the tab badges.

Lines	Code	What it does
465–467	<pre>function openModal(modalId) { document.getElementById(modalId).classList.remove('hidden'); }</pre>	<code>openModal(modalId)</code> finds the popup by its id and removes <code>hidden</code> , so it appears.
469–471	<pre>function closeModal(modalId) { document.getElementById(modalId).classList.add('hidden'); }</pre>	<code>closeModal(modalId)</code> adds <code>hidden</code> , so it disappears.
473–477	<pre>function resetPortalData() { if (confirm("Reset jobs, bookmarks and applications back to the demo data? Your account will be kept.)) { localStorage.removeItem('hp_jobs'); localStorage.removeItem('hp_applications'); if (currentUser) localStorage.removeItem('hp_savedJobs_' + currentUser.id); } }</pre>	<code>resetPortalData()</code> asks for confirmation (<code>confirm</code>), then removes the saved jobs and applications, and this user's bookmarks if someone is logged in (<code>removeItem</code>).
478–485	<pre>appState.jobs = SEED_JOBS.map(j => ({ ...j })); appState.savedJobIds = []; appState.applications = []; syncStorage(); showToast("Portal data reset to default seed data."); render(); }</pre>	Puts the demo jobs back: <code>SEED_JOBS.map(j => ({ ...j }))</code> makes fresh copies (<code>...</code> spreads an object's fields). Clears bookmarks and applications, saves, shows a toast and redraws. The account is kept.
487–490	<pre>function showToast(message, type = 'success') { const container = document.getElementById('toastContainer'); const toast = document.createElement('div'); toast.className = 'toast';</pre>	<code>showToast(message, type = 'success')</code> : <code>type = 'success'</code> is a default value. It creates a new <code>div</code> with <code>createElement</code> , gives it the class <code>toast</code> .
491–492	<pre>toast.innerHTML = `<i><i class="fa-solid fa-circle-info"></i></i> \${escapeHTML(message)}`; container.appendChild(toast);</pre>	Sets its inner HTML to an icon plus the (escaped) message, then adds it to the toast container.
494–497	<pre>setTimeout(() => { toast.remove(); }, 3000);</pre>	<code>setTimeout(() => { toast.remove(); }, 3000)</code> removes the toast after 3000 milliseconds (3 seconds).
499–504	<pre>function escapeHTML(str) { if (!str) return ''; return str.replace(/[&<>"]/g, tag => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&#39;', "'": '&quot;' }[tag] tag)); }</pre>	<code>escapeHTML(str)</code> protects against injected code. If <code>str</code> is empty it returns <code>''</code> . Otherwise <code>.replace(/[&<>"]/g, ...)</code> uses a regular expression to find each dangerous character and swaps it for its safe HTML code (for example <code><</code> becomes <code>&lt;</code>), so typed text is shown as text and never run as HTML.

Section 7: Authentication (login / sign up)

Lines	Code	What it does
506–511	<pre>// ===== // 7. AUTHENTICATION (LOGIN / SIGN UP) // ===== // NOTE: This is a front-end demo. Accounts live in this browser's // localStorage, // so it is NOT real security. A production portal needs a backend with a // proper // auth service (hashed passwords server-side, sessions/JWT, HTTPS).</pre>	A banner and a note: this login is a front-end demo that stores accounts in the browser, so it is not real security. A real site needs a backend with server-side password hashing and sessions.
513–514	<pre>const USERS_KEY = 'hp_users'; const SESSION_KEY = 'hp_session';</pre>	Two constants holding the localStorage key names for the user list and the current session.
516–519	<pre>function getUsers() { try { return JSON.parse(localStorage.getItem(USERS_KEY)) []; } catch (err) { return []; } }</pre>	<code>getUsers()</code> returns the saved user list. <code>try { ... } catch (err) { return []; }</code> means "try this; if it fails (for example corrupted data), return an empty list instead of crashing".
521–523	<pre>function saveUsers(users) { localStorage.setItem(USERS_KEY, JSON.stringify(users)); }</pre>	<code>saveUsers(users)</code> stores the list as text.
525–532	<pre>function makeSalt() { if (window.crypto && crypto.getRandomValues) { const bytes = new Uint8Array(16); crypto.getRandomValues(bytes); return Array.from(bytes, b => b.toString(16).padStart(2, '0')).join(''); } return String(Date.now()) + Math.random().toString(16).slice(2); }</pre>	<code>makeSalt()</code> creates a random string (a "salt") so identical passwords get different hashes. <code>crypto.getRandomValues</code> fills a <code>Uint8Array(16)</code> with 16 random bytes; <code>.toString(16).padStart(2, '0')</code> converts each to two hex characters; <code>.join('')</code> joins them. The last line is a fallback if crypto is unavailable.
534–535	<pre>async function hashPassword(password, salt) { const text = salt + password;</pre>	<code>async function hashPassword(password, salt)</code> is an asynchronous function. It combines salt and password into one text.
536–539	<pre> if (window.crypto && crypto.subtle) { const buf = await crypto.subtle.digest('SHA-256', new TextEncoder().encode(text)); return Array.from(new Uint8Array(buf), b => b.toString(16).padStart(2, '0')).join(''); }</pre>	<code>crypto.subtle.digest('SHA-256', ...)</code> computes a SHA-256 hash (a one-way scrambled fingerprint). <code>await</code> pauses until the result is ready. The bytes are converted to a hex string.
540–542	<pre> // Fallback for non-secure contexts where crypto.subtle is unavailable return btoa(unescape(encodeURIComponent(text))); }</pre>	A weaker fallback (<code>btoa</code>) for browsers where <code>crypto.subtle</code> is not available, such as plain http pages. Then <code>}</code> ends the function.
544–547	<pre>function myApplications() { if (!currentUser) return []; return appState.applications.filter(a => (a.email '').toLowerCase() === currentUser.email); }</pre>	<code>myApplications()</code> returns an empty list if nobody is logged in; otherwise only the applications whose email matches the current user's email.

Lines	Code	What it does
549–553	<pre>function showAuthError(message) { const box = document.getElementById('authError'); box.textContent = message; box.classList.remove('hidden'); }</pre>	<code>showAuthError(message)</code> writes the message into the red box and shows it.
555–557	<pre>function clearAuthError() { document.getElementById('authError').classList.add('hidden'); }</pre>	<code>clearAuthError()</code> hides the red box.
559–566	<pre>function switchAuthTab(tab) { const isLogin = tab === 'login'; document.getElementById('loginForm').classList.toggle('hidden', !isLogin); document.getElementById('signupForm').classList.toggle('hidden', isLogin); document.getElementById('tabLogin').classList.toggle('active', isLogin); document.getElementById('tabSignup').classList.toggle('active', !isLogin); clearAuthError(); }</pre>	<code>switchAuthTab(tab)</code> : <code>isLogin</code> is true or false. <code>classList.toggle('hidden', !isLogin)</code> adds or removes <code>hidden</code> depending on the second value (! means NOT), so only one form shows, and the matching tab gets <code>active</code> . Old errors are cleared.
568–574	<pre>function togglePassword(inputId, btn) { const input = document.getElementById(inputId); const show = input.type === 'password'; input.type = show ? 'text' : 'password'; btn.innerHTML = `<i class="fa-regular fa-eye\${show ? '-slash' : ''}"></i>`; btn.setAttribute('aria-label', show ? 'Hide password' : 'Show password'); }</pre>	<code>togglePassword(inputId, btn)</code> flips the field between <code>type="password"</code> and <code>type="text"</code> , changes the icon to eye or eye-slash, and updates the button's <code>aria-label</code> .
576–578	<pre>async function handleSignup(e) { e.preventDefault(); clearAuthError(); }</pre>	<code>async function handleSignup(e)</code> stops the page reload and clears old errors.
580–584	<pre>const name = document.getElementById('signupName').value.trim(); const email = document.getElementById('signupEmail').value.trim().toLowerCase(); const role = document.querySelector('input[name="signupRole"]:checked').value; const password = document.getElementById('signupPassword').value; const confirmPassword = document.getElementById('signupConfirm').value;</pre>	Reads the form values. <code>document.querySelector('input[name="signupRole"]:checked').value</code> finds the selected radio button and gets its value ("seeker" or "employer").
586–588	<pre>if (name.length < 2) return showAuthError('Enter your full name.');</pre> <pre>if (password.length < 8) return showAuthError('Password must be at least 8 characters.');</pre> <pre>if (password !== confirmPassword) return showAuthError('Passwords do not match. Re-enter them and try again.');</pre>	Validation. Each <code>if</code> checks a rule and <code>return showAuthError(...)</code> shows the message and stops the function.

Lines	Code	What it does
590–593	<pre>const users = getUsers(); if (users.some(u => u.email === email)) { return showAuthError('An account with this email already exists. Log in instead.');</pre>	<code>users.some(u => u.email === email)</code> checks whether this email is already registered.
595–604	<pre>const salt = makeSalt(); const user = { id: 'u' + Date.now(), name, email, role, salt, passwordHash: await hashPassword(password, salt), createdAt: new Date().toISOString() };</pre>	Makes a salt and builds the <code>user</code> object: unique id (<code>'u' + Date.now()</code>), name, email, role, salt, the hashed password (<code>await hashPassword(...)</code>) and the creation time (<code>toISOString()</code>). The plain password is never stored.
605–608	<pre>users.push(user); saveUsers(users); localStorage.setItem(SESSION_KEY, user.id);</pre>	Adds the user to the list, saves the list and stores the session id so the user stays logged in after a refresh.
609–612	<pre>document.getElementById('signupForm').reset(); enterApp(user); showToast(`Welcome to HirePulse, \${name.split(' ')[0]}!`); }</pre>	Resets the form, calls <code>enterApp(user)</code> to open the portal and shows a welcome toast. <code>name.split(' ')[0]</code> takes the first word of the name.
614–616	<pre>async function handleLogin(e) { e.preventDefault(); clearAuthError();</pre>	<code>async function handleLogin(e)</code> stops reload and clears errors.
618–620	<pre>const email = document.getElementById('loginEmail').value.trim().toLowerCase(); const password = document.getElementById('loginPassword').value; const user = getUsers().find(u => u.email === email);</pre>	Reads email and password, then <code>getUsers().find(u => u.email === email)</code> looks for the matching user.
622–624	<pre>if (!user (await hashPassword(password, user.salt)) !== user.passwordHash) { return showAuthError('Incorrect email or password.');</pre>	If no user is found, or the new hash does not equal the stored <code>passwordHash</code> , show one generic error. It does not reveal which part was wrong (safer).
626–630	<pre>localStorage.setItem(SESSION_KEY, user.id); document.getElementById('loginForm').reset(); enterApp(user); showToast(`Welcome back, \${user.name.split(' ')[0]}!`); }</pre>	On success: save the session, reset the form, open the app and show "Welcome back".

Lines	Code	What it does
632–640	<pre>function enterApp(user) { currentUser = user; appState.activeRole = user.role; appState.activeSeekerTab = 'all'; try { appState.savedJobIds = JSON.parse(localStorage.getItem('hp_savedJobs_' + user.id)) []; } catch (err) { appState.savedJobIds = []; } }</pre>	<code>enterApp(user)</code> stores the user, sets the role and tab, and loads this user's bookmarks from <code>localStorage</code> (inside <code>try/catch</code> in case the data is damaged).
642–644	<pre>document.getElementById('authScreen').classList.add('hidden'); document.getElementById('appHeader').classList.remove('hidden'); document.getElementById('appMain').classList.remove('hidden');</pre>	Hides the auth screen and shows the navbar and the main area.
646–648	<pre>document.getElementById('userAvatar').textContent = user.name.charAt(0).toUpperCase(); document.getElementById('userNameLabel').textContent = user.name; document.getElementById('userRoleLabel').textContent = user.role === 'employer' ? 'Employer / Recruiter' : 'Job Seeker';</pre>	Fills the navbar chip: first letter in capitals (<code>charAt(0).toUpperCase()</code>), the name, and the role text using a ternary.
650–654	<pre>document.querySelectorAll('.tab-btn').forEach(btn => btn.classList.remove('active')); document.getElementById('tabAllJobs').classList.add('active'); document.getElementById('searchInput').value = ''; document.getElementById('categoryFilter').value = 'all'; document.getElementById('typeFilter').value = 'all';</pre>	Resets the tab to All Listings, clears the search box and sets both filters back to "all".
656–657	<pre>render(); }</pre>	<code>render()</code> draws the right screen and <code>}</code> ends the function.
659–665	<pre>function showAuthScreen() { currentUser = null; document.getElementById('authScreen').classList.remove('hidden'); document.getElementById('appHeader').classList.add('hidden'); document.getElementById('appMain').classList.add('hidden'); switchAuthTab('login'); }</pre>	<code>showAuthScreen()</code> clears the current user, shows the login screen, hides the navbar and main area and selects the login tab.
667–672	<pre>function restoreSession() { const sessionId = localStorage.getItem(SESSION_KEY); const user = sessionId ? getUsers().find(u => u.id === sessionId) : null; if (user) enterApp(user); else showAuthScreen(); }</pre>	<code>restoreSession()</code> reads the saved session id and looks up the user. A short <code>if / else</code> either opens the app (<code>enterApp</code>) or shows the login screen.

Lines	Code	What it does
674–680	<pre>function handleLogout() { localStorage.removeItem(SESSION_KEY); appState.savedJobIds = []; document.querySelectorAll('.modal-overlay').forEach(m => m.classList.add('hidden')); showAuthScreen(); showToast('You have been logged out.');</pre>	<code>handleLogout()</code> removes the session, clears the bookmarks in memory, hides any open popups, shows the login screen and displays a "logged out" toast.